Wiki COALA Python

Maxime Lombart

12 October 2025

CONTENTS

1 INTRODUCTION

The code COALA is a high-order scheme based on the discontinuous Galerkin (DG) method to solve the Smoluchowski coagulation equation in its conservative form (see details in Lombart & Laibe 2021). This new scheme allows to treat accurately dust coagulation in 3D simulations.

Current version:

- Only coagulation
- DG scheme with integrals calculated with Gauss-Legendre quadrature method
- Any collision kernels can be used

2 DG SCHEME APPLIED ON THE COAGULATION EQUATION

The conservative form of the coagulation equation on physical mass domain writes

$$\begin{cases} \dfrac{\partial g(m,t)}{\partial t} + \dfrac{F_{\rm coag}[g](m,t)}{\partial m}, \\ F_{\rm coag}[g](m,t) \equiv \displaystyle\int_{m_{\rm min}}^{m} \int_{m-m_1+m_{\rm min}}^{m_{\rm max}-m_1+m_{\rm min}} \frac{K(m_1,m_2)}{m_2} g(m_1,t) g(m_2,t) {\rm d}m_2 {\rm d}m_1, \end{cases} \tag{1}$$

where $m \in [m_{\rm min}, m_{\rm max}]$ is the mass of dust grains, t the time, g the mass density distribution (cm^{-3} in cgs), K the collision kernel encoding the physics of the grain-grain collision ($cm^3.s^{-1}$ in cgs). $F_{\rm coag}$ is denoted the coagulation flux.

The mass range is discretized in N bins and the DG method is applied on Eq. 1 to obtain the following equation

$$\int_{I_j} \frac{\partial g_j(m,t)}{\partial t} \phi\left(\xi_j(m)\right) \, {\rm d}m - \int_{I_j} F_{\rm coag}[g](m,t) \partial_m \phi\left(\xi_j(m)\right) \, {\rm d}m + \left[F_{\rm coag}[g](m,t) \phi\left(\xi_j(m)\right)\right]_{I_j} = 0, \tag{2}$$

where $I_j \equiv [m_{j-1/2}, m_{j+1/2}]$ is the boundary of bin j, ϕ are chosen as the Legendre polynomials. Function g_j is the approximation of g on the Legendre polynomials basis in bin j such as

$$\forall m \in I_j, \; g(m,t) \approx g_j(m,t) = \sum_{i=0}^{k} g_j^i(t) \phi_i\left(\xi_j(m)\right), \tag{3}$$

where k is the order of the polynomial, g_j^i are the components in the polynomial basis. By injecting the expression of g_j,

we obtain the following system to solve

$$\begin{cases} \forall j \in [1, N], \forall i \in [0, k], \\ \dfrac{\mathrm{d}g_j^i(t)}{\mathrm{d}t} = L[g](j, i, t), \\ g_j^i(0) = \dfrac{1}{d_i} \displaystyle\int_{I_j} g(m, 0) \phi_i \left(\xi_j(m)\right) \, \mathrm{d}m, \end{cases} \tag{4}$$

where the initial condition $g_j^i(0)$ is obtained by piecewise L_2 projection on the polynomials basis, d_i is the normalisation coefficient of Legendre polynomials and the operator L writes

$$L[g](j, i, t) \equiv \frac{2}{h_j d_i} \left(\underbrace{\int_{I_j} F_{\mathrm{coag}}[g](m, t) \partial_m \phi_i \left(\xi_j(m)\right) \, \mathrm{d}m}_{intF_{\mathrm{coag}}} - \left[F_{\mathrm{coag}}[g](m, t) \phi_i \left(\xi_j(m)\right) \right]_{I_j} \right). \tag{5}$$

The main difficulty is to evaluate accurately the operator L, which depends on the evaluation of all the integrals. Thanks to the DG method, variables m and t are separated which allows to precompute the integrals depending only on mass variable to keep fast algorithm. The expression of the flux and the term with integral of the flux are given by Eqs. 25-26 in Lombart & Laibe (2021):

$$\begin{aligned} F_{\mathrm{coag}}[g](m_{j+1/2}, t) &\approx \sum_{l'=1}^{j} \sum_{i'=0}^{k} \sum_{l=1}^{N} \sum_{i=0}^{k} g_{l'}^{i'}(t) g_l^i(t) \; T(m_{\max}, m_{\min}, j, l', l, i', i), \\ intF_{\mathrm{coag}}[g](j, k', t) &\approx \sum_{l'=1}^{j} \sum_{i'=0}^{k} \sum_{l=1}^{N} \sum_{i=0}^{k} g_{l'}^{i'}(t) g_l^i(t) \; intT(m_{\max}, m_{\min}, j, k', l', l, i', i). \end{aligned} \tag{6}$$

The precalculation part is to compute the arrays with element $T(m_{\max}, m_{\min}, j, l', l, i', i)$ and $intT(m_{\max}, m_{\min}, j, k', l', l, i', i)$.

The `COALA` algorithm solves this ODE equation by applying a Strong Stability Preserving Runge-Kutta order 3 (see details in Lombart & Laibe 2021).

In the description of the python code, N si given by `nbins` and k is given by `kpol`.

3 REQUIREMENTS

`COALA` requires python packages. High performances of the precomputation part is reached with Numba. The required python packages are (see requirements.txt)

```
numpy
numba
numba_progress
scipy
matplotlib
progressbar
mpmath
```

4 COALA AS LOCAL PYTHON PACKAGE

The python version of `COALA` is build to be used as local python package. First, it is required to add the path into the python environment :

```
export PYTHONPATH=/your_absolute_path_to_coala_directory/
```

Then, `COALA` source files are imported with the following command

```
from coala_py.src import *
```

5 COALA SOURCE FILES

The source files of the code are located in `src/`:

`compute_coag.py`: compute coagulation solver for 1 hydro timestep

`exact_solutions_coag.py`: functions to compute exact solutions of the Smoluchowski equation for simple kernels

`generate_flux_intflux.py`: functions to compute the approximation of the coagulation flux with DG scheme

`generate_tabflux_tabintflux.py`: functions to precompute arrays depending only on massgrid to evaluate the coagulation flux and the term including the integral of flux, i.e. Eq. 6

`init_massgrid.py`: generate grid in mass in logscale (to span several orders of magnitude in mass with few number of bins)

`iterate_coag.py`: functions to iterate coagulation solver to reach the time `ndthydro * dthydro`

`kernel_collision.py`: file containing the kernel functions (any kernels can by added through a function)

`L2_proj.py`: functions to compute the initial g_j^i coefficient from the initial function `g(m) = m*exp(-m)`

`limiter.py`: compute the limiter coefficient to ensure positivity of the numerical solution (Zhang & Shu 2010)

`reconstruction_g.py`: function to reconstruct the approximation of g using the components g_j^i and the Legendre polynomial basis

`utils_polynomials.py`: functions to evaluate Legendre polynomials and their derivative

`/coag_term/coag_functions_GQ.py`: functions to evaluate the integrand of the coagulation flux, i.e. integrand of the double integral T, and the integrand of the term including the integral of the flux, i.e. triple integral $intT$

6 COALA TESTS

The code `COALA` can be tested with two different tests. The first one is the comparaison with the analytical solutions for the Smoluchowski coagulation equation (constant and additive kernels). The second test is with a physical kernel with the Brownian motion as source of differential velocity between grains. All the tests are performed with the file `test_coag/coala_coag.py`. This file includes the following parameters required for `COALA`

```
# setup parameters
#-----------------------------------------
# nbins     -> number of grain size bins
# massmax   -> maximal mass of grains to consider
# minmass   -> minimal mass of grains to consider
# kernel    -> chose among available kernels (see kernel_collision.py)
# K0        -> normalisation coefficient for simple kernels
# kpol      -> order of polynomials for approximation
# Q         -> number of Gauss points for Gauss-quadrature method
# eps       -> minimum value for mass density distribution
# coeff_CFL -> coefficient for SSPRK 3 time solver
# dthydro   -> hydro timestep
# ndthydro  -> number of dthydro
#-----------------------------------------
```

6.1 Tests simple kernels

Only tests with constant and additive kernels are available to compare numerical solutions to the exact solutions.

6.1.1 Setup

- Choose the limits of the mass range `massmin` and `massmax` (dimensionless) and the number of `nbins` for discetization. The choice of the mass range is coherent with the initial condition `g(m)=m*exp(-m)` from which the maximum value is located at `m=1`. The default values are

```
nbins = 20

```

```
massmax = 1e6
massmin = 1e-3
```

- Choose among the simple kernels with the parameter `kernel` (0 for constant, 1 for additive). For these kernels, the constant coefficient `K0=1`.
- Choose the order of polynomials for approximation with the parameter `kpol`. Default values is `kpol=0`.
- The number of Gauss points is defined by `Q`, default value is 15 in order to evaluate accurately integrals. Parameter `eps = 1e-20` defines the minimum authorized value for the polynomial approximation of the function g. The CFL coefficient is set to `coeff_CFL = 0.3` for the Strong Stability Preserving Runge-Kutta order 3 method.
- Hydrodynamic timestep `dthydro` and the number of timesteps `ndthydro` are set in dimensionless specifically for each kernel, to be compatible with the exact solutions, as following

```
#pre defined values for dthydro and ndthydro

#constant kernel
dthydro = 100
ndthydro = 300

#additive kernel
dthydro = 1e-2
ndthydro = 300
```

6.1.2 Initial mass distribution

The initial mass distribution for the simple kernels is $g(m, 0) = mexp(-m)$ with m the dimensionless mass. Functions to generate the initial mass distribution g are available in `L2_proj.py`:

```
#initial mass distribution for DG scheme order 0
gij = L2projDL_k0(eps,nbins,massgrid,massbins,Q,vecnodes,vecweights)

#initial mass distribution for DG scheme higher orders
gij = L2projDL(eps,nbins,kpol,massgrid,massbins,Q,vecnodes,vecweights)
```

6.1.3 Run tests

All the steps of the DG scheme are written in the file `src/iterate_coag.py`. To run `COALA` with specific parameters, run the python file `coala_coag.py`. All data will saved locally in the directory `./data/`.

6.1.4 Plots

The python file `plot_simple_kernels.py` read the data to plot the mass density distribution and compare the numerical to the exact solutions. Three parameters are required to read the data:

```
kernel = 0
nbins  = 20
Q      = 15
```

At the end of the file, you can choose to save the plot or display it. By default the plot is displayed.

6.2 Test physical kernel

The physical kernel is called the ballistic kernel and writes $K(x, y) = \sigma(x, y)\Delta v(x, y)$, with $\sigma(x, y)$ the cross-section and Δv the differential velocity between grains. If the analytical expression of $\Delta v(x, y)$ is not known, only the cross-section is used to evaluate the pre-calculations for the flux and integral of the flux. An approximation of the differential velocity is used in the DG scheme as a 2D histogram, then just implemented by multiplication to calculate flux and integral of the flux. This process mimic a coupling between `COALA` and any hydro solver, i.e. when Δv is obtained from hydrodynamics. The python file `coala_coag.py` is also used for the physical kernel test.

6.2.1 Setup

The input parameters are similar to the simple kernels test setup (see Sect. 6.1.1). One parameter differs, the choice of the kernel. Here only two values are possible as following

```
# 2 -> brownian motion kernel with analytical dv : K = sigma * dv
# 3 -> kdv with analytical cross section, need approximated dv (2D array)
kernel = 2 #(or 3)
```

For `kernel = 2`, the analytical expression of the differential velocity from Brownian motion is used, in order to obtain a reference solution. In that case, the complete expression of the physical kernel is used to pre-compute the integrals. The reference solution is defined with `nbins = 100` and `kpol = 0`.

For `kernel = 3`, only the approximation of $\Delta v(x, y)$ is known (2D histogram). Therefore, only the cross-section is used for precalculations and the 2D histogram is used to evaluated the flux and integral of the flux.

Only Brownian motion physical kernel is available.

6.2.2 Initial mass distribution

The initial mass distribution for the physical kernel is $g(m, 0) = mexp(-m)$ with m the dimensional mass, similar to the tests with simple kernels.
Initial condition with power law will be added in a future.

6.2.3 Run test

Run the python file `coala_coag.py`. All data will saved locally in the directory `./data/`. For `kernel = 2`, data are saved in `./data/dv_brownian_ref/`. For `kernel = 3`, data are saved in `./data/dv_brownian_approx/`.

6.2.4 Plot

The python file `plot_dv_brownian.py` read the data to plots the mass density distribution and compare the numerical and the reference solutions. Three parameters are required to read the data:

```
nbins_ref = 100
nbins     = 20
Q         = 15
```

At the end of the file, you can choose to save the plot or display it. By default the plot is displayed.

7 HOW TO USE COALA IN YOUR CODE

7.1 Link between COALA and hydro solver

The dust coagulation process is a source term of the continuity equation:

$$\forall j \in [1, N],\ \frac{\partial \rho_j}{\partial t} + \nabla \cdot (\rho_j \mathbf{v}_k) = \left.\frac{\partial \rho_j}{\partial t}\right|_{\text{coag}}, \tag{7}$$

where the link between the dust mass density ρ_j and the mass density distribution g is

$$\rho_j = \int_{I_j} g(m, t)\, \mathrm{d}m. \tag{8}$$

7.1.1 Constant piecewise approximation

When constant piecewise approximation is used for g, we directly have the link $\overline{g}_j(t) = g_j^0(t) = \frac{\rho_j}{h_j}$. Then, the ODE to solve is

$$\frac{\mathrm{d}g_j^0(t)}{\mathrm{d}t} = -\frac{2}{h_j d_0}\left(F_{\text{coag}}[g](m_{j+1/2}, t) - F_{\text{coag}}[g](m_{j-1/2}, t)\right) \tag{9}$$

7.1.2 Polynomial piecewise approximation

When g is approximated by polynomials in each bin j, it is necessary to interpolate the cumulative function $C(m) \equiv \int_{m_{\min}}^{m} g(m',t)\mathrm{d}m$, in order to take its derivative to obtain $g(m,t)$. The interpolation points are obtained on the mass grid such as

$$C(m_{j+1/2}) = \sum_{i=1}^{j} \rho_j. \tag{10}$$

The main difficulty here is to have an accurate interpolation, the accuracy of the global algorithm depends on it. When the continuous function $g(m,t)$ is obtained in each, this function is approximated on the Legendre polynomial basis to get g_j^i and the ODE to solve is

$$\forall j \in [1,N], \forall i \in [0,k],\ \frac{\mathrm{d}g_j^i(t)}{\mathrm{d}t} = L[g](j,i,t) \tag{11}$$

Coupling `COALA` with hydro code

The coupling depends on which time solver is used to solve the ODE. Two ways are possible:

(i) with the hydro time solver
(ii) with the time solver implemented in `COALA`

In case i), the operator L has to be evaluated in the hydro code, meaning that the terms F_{coag} and $intF_{\mathrm{coag}}$ need to be calculated in the hydro code for easy coupling. In case ii), all the functions are in the code `COALA`, the evolved ρ_j is given after 1 hydro timestep.

In both cases, precalculated arrays are needed to evaluate all the terms depending on the coagulation flux. The precomputating functions are provided by `COALA`. These functions requires to define the following parameters, given with the default values

```
nbins      = 20
kernel     = 3 # -> only cross-section in mass variable
# K0 -> to be used to adapt to code unit
kpol       = 0
Q          = 15
```

7.2 Generate `massgrid` and `massbins`

The mass grid is the main input parameter for precalculations.
It is assumed that the variables `massgrid` (the mass grid → boundaries of mass bins) and `massbins` (arithmetic mean of the mass grid) can be provided by the hydro solver.
If it not the case, the function `init_grid_log(nbins,massmax,massmin)` in `src/init_massgrid.py` can be used to compute `massgrid, massbins` by giving the following variables

```
massmax #mass of the maximum grain size considered in the simulation
massmin #mass of the minimum grain size considered in the simulation
```

7.3 Pre-calculations

COALA is designed to be as fast as possible thanks to the pre-calculation part (functions depending only on the mass grid) for the time solver. The pre-calculation is done only once for one simulation.
Precalculation is the evaluation of the integrals depending on the coagulation flux. The integrals are performed with the Gauss-Legendre quadrature method. Therefore, it is required to get the nodes and weights with the following function

```
import numpy as numpy
vecnodes,vecweights = np.polynomial.legendre.leggauss(Q)
```

Then the Numba functions to precalculate the arrays are in `generate_tabflux_tabintflux.py`:

```
#for kpol = 0
tensor_tabflux_coag = np.zeros((nbins,nbins,nbins))
compute_coagtabflux_numba(kernel,K0,Q,vecnodes,vecweights,nbins,kpol,massgrid,
    mat_coeffs_leg,tensor_tabflux_coag,progress)

#for kpol > 0
```

```
tensor_tabflux_coag = np.zeros((nbins,nbins,nbins,kpol+1,kpol+1))
compute_coagtabflux_numba(kernel,K0,Q,vecnodes,vecweights,nbins,kpol,massgrid,
    mat_coeffs_leg,tensor_tabflux_coag,progress)

tensor_tabintflux_coag = np.zeros((nbins,kpol+1,nbins,nbins,kpol+1,kpol+1))
compute_coagtabintflux_numba(kernel,K0,Q,vecnodes,vecweights,nbins,kpol,massgrid,
    mat_coeffs_leg,tensor_tabintflux_coag,progress)
```

The parameter `mat_coeffs_leg` is an array giving the coefficients of Legendre polynomials up to order `kpol`. This array is obtained with the function `legendre_coeffs(kpol)` in `utils_polynomials.py`.

7.4 Use time solver from hydro code

I detail here the steps for the DG scheme in hydro code.

7.4.1 Constant piecewise approximation

(i) initial coefficients
The coefficients are obtained by $g_j^0 = \rho_j / h_j$.

(ii) Evaluate flux
The flux is calculated by including the 2D array for the differential velocity given by the hydro code

$$\forall j \in [1,N],\ F_j \equiv F_{\rm coag}[g](m_{j+1/2},t) \approx \sum_{l'=1}^{j}\sum_{l=1}^{N} g_{l'}^0(t) g_l^0(t)\ T(m_{\rm max}, m_{\rm min}, j, l', l, 0, 0)\Delta v_{l',l} \tag{12}$$

We force the left boundary condition for the flux at $m_{\rm min}$, meaning that $F_0 \equiv F_{\rm coag}[g](m_{\rm min},t) = 0$.

(iii) Coagulation CFL
The CFL condition is determined like in the paper Filbet & Laurencot (2004):

$$dt_{\rm CFL} = \min_j \left(\frac{g_j^0 h_j}{|F_j - F_{j-1}|} \right) \tag{13}$$

(iv) Solve ODE
If $dt_{\rm hydro} < dt_{\rm CFL}$, then the ODE is solved, for instance with Euler-Forward method, as

$$\forall j \in [1,N],\ g_j^{0,n+1} = g_j^{0,n} - dt_{\rm hydro} \frac{2}{h_j d_0} \left(F_j - F_{j-1}\right). \tag{14}$$

If $dt_{\rm CFL} < dt_{\rm hydro}$, it requires several coagulation timesteps to reach $dt_{\rm hydro}$. The pseudo code is the following:

$dt = dt_{\rm CFL}$
while $dt < dt_{\rm hydro}$ **do**
  $\forall j \in [1,N],\ g_j^{0,m+1} = g_j^{0,m} - dt \frac{2}{h_j d_0} \left(F_j - F_{j-1}\right)$
  F_j calculated with $g_j^{0,m+1}$
  $dt = dt + \min_j \left(\frac{g_j^{0,m+1} h_j}{|F_j - F_{j-1}|} \right)$
  $m = m + 1$
end while
$dt_{\rm last} = dt_{\rm hydro} - dt \rightarrow \forall j \in [1,N],\ g_j^{0,n+1} = g_j^{0,m+1} - dt_{\rm last} \frac{2}{h_j d_0} \left(F_j - F_{j-1}\right)$

Then, the evolved dust density is given by $\rho_j^{n+1} = h_j g_j^{0,n+1}$

Polynomial piecewise approximation
This part will be detailed in a future.

REFERENCES

Filbet F., Laurencot P., 2004, SIAM Journal on Scientific Computing, 25, 2004
Lombart M., Laibe G., 2021, MNRAS, 501, 4298
Zhang X., Shu C.-W., 2010, Journal of Computational Physics, 229, 3091